



Отчет по лабораторной работе

Задание 1 по курсу «Информационные технологии» вариант 11

Учебная группа

М7О-207БВ-24

Студент

Никитин П.А. _____

Преподаватель

Барчев Н.Б. _____

Содержание

1 Текст задания лабораторной работы	1
2 Псевдокод	2
3 Сведения о программной реализации	5
3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования	5
3.2 Описание входных и выходных данных	5
3.3 Краткое описание спроектированных программных единиц	6
3.4 Список диагностических сообщений с указанием возможных причин их появления	9
4 Инструкция использования программы	10
5 Листинг программной разработки	12
6 Результаты тестирования	20

1

Текст задания лабораторной работы

Задание 1 по курсу «Информационные технологии» вариант 11

1. Подготовить программу, формирующую на основе информации, вводимой пользователем с клавиатуры, два внешних текстовых файла:
 - файл автомобилей: состоит из записей, каждая из которых включает три поля – марки, модели и номерного знака;
 - файл регистрационных сведений: состоит из записей, каждая из которых включает четыре поля – номерного знака, фамилии и адреса владельца, года выпуска.
2. В процессе проектирования предусмотреть необходимые по смыслу задания проверки корректности данных, а также адекватное задаче взаимодействие с пользователем. **Учесть, что ограничения на размеры файлов отсутствуют, файлы в общем случае могут иметь различные длину.**
В обязательном порядке использовать языковые средства организации программных единиц.
Не использовать программные средства, поддерживающие работу с базами данных.
3. Программу подготовить с использованием средств одной из реализаций языка программирования C++, **ввод и вывод информации реализовать при помощи средств языка программирования С.** Выполнить тестирование обеих программ согласно стратегии черного ящика, используя соответствующие критерии тестирования.

2

Псевдокод

файл main.cpp:

```
1 ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"
2 ВЫЗОВ функции main()
3
4 функция main():
5     УСТАНОВИТЬ ЛОКАЛЬ: en_RU.UTF-8
6     ВЫВЕСТИ: Назначение программы
7     ВЫВЕСТИ: Возможные опции
8     ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ВВЕЛ EOF:
9         ОБРАБОТАТЬ ПОТОК ВВОДА ДЛЯ ВЗАИМОДЕЙСТВИЯ С ПРОГРАММОЙ
10    ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ РАБОТУ С ПРОГРАММОЙ ТО:
11        ВЫЙТИ ИЗ ЦИКЛА
12    ОЖИДАТЬ ПОЛЬЗОВАТЕЛЬСКИЙ ВВОД
13    ВЫВЕСТИ: Сообщение о том, что программа завершена.
14    ЗАВЕРШИТЬ ПРОГРАММУ
15    ВЕРНУТЬ 0
```

файл file_handler.cpp:

```
1 ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"
2
3 ПРОИНИЦИАЛИЗИРОВАТЬ глобальные переменные: СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1,
  СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2, НИЖНЯЯ_ГРАНИЦА
4
5 ОБЪЯВИТЬ БУФФЕР[256] // ИЗ 256 ЭЛЕМЕНТОВ
6
7 функция clear_stdin():
8     ОЧИЩАЕТ ПОТОК ВВОДА
9
10 функция validate_filename(имя_файла):
11     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_файла{"[a-zA-Z0-9_-]+\.\txt"}
12     ВЕРНУТЬ имя_файла СООТВЕТСТВУЕТ регулярное_выражение_файла
13
14 функция validate_license(номерной_знак):
15     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_номерного_знака{"[АВЕКМНОРСТУХ][0-9]{3,3}
  [АВЕКМНОРСТУХ]{2,2}[0-9]{2,3}RUS"}
16     ВЕРНУТЬ номерной_знак СООТВЕТСТВУЕТ регулярное_выражение_номерного_знака
17
18 функция validate_brand(бренд):
19     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_бренда{"[А-ЯЁа-яёА-Za-z0-9-_.! ]+"}
```

```
20  ВЕРНУТЬ бренд СООТВЕТСТВУЕТ регулярное_выражение_бренда
21
22 функция validate_model(модель):
23  ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_модели{"[А-ЯЁа-яёА-Za-z0-9-_.!() ]+"}
24  ВЕРНУТЬ модель СООТВЕТСТВУЕТ регулярное_выражение_модели
25
26 функция validate_surname(фамилия):
27  ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_фамилии{"[А-ЯЁа-яёА-Za-z]+"}
28  ВЕРНУТЬ фамилия СООТВЕТСТВУЕТ регулярное_выражение_фамилии
29
30 функция validate_address(адрес):
31  ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_адреса{"[А-Za-zА-ЯЁа-яё0-9!_-. ]"}
32  ВЕРНУТЬ адрес СООТВЕТСТВУЕТ регулярное_выражение_адреса
33
34 функция validate_release_year(год_выпуска):
35  ПРОИНИЦИАЛИЗИРОВАТЬ результат = ИСТИНА
36  ЕСЛИ год_выпуска ПУСТОЙ ИЛИ СОДЕРЖИТ НЕЦИФРЫ ТО:
37    ВЫВЕСТИ: Сообщение о неправильном вводе.
38    ВЕРНУТЬ ЛОЖЬ
39
40  ПРОИНИЦИАЛИЗИРОВАТЬ текущий_год = ТЕКУЩИЙ ГОД СИСТЕМЫ
41  ЕСЛИ год_выпуска < НИЖНЯЯ_ГРАНИЦА ИЛИ год_выпуска > ТЕКУЩИЙ ГОД СИСТЕМЫ ТО:
42    ВЫВЕСТИ: "Год вне диапазона"
43    результат = ЛОЖЬ
44  ВЕРНУТЬ результат
45
46 функция file_info(стандартное_имя_файла):
47  ПРИНЯТЬ пользовательский ввод
48  ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ имя_файла
49  ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ режим_открытия_файла
50  ВЕРНУТЬ имя_файла И режим_открытия_файла
51 функция write_car_data():
52  ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла =
file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1)
53  ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла
54
55  ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:
56    ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
57    ВЕРНУТЬ
58  ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"
59
60  ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:
61    ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:
62      ВЫВЕСТИ: Сообщение об ошибке в введенном поле
63      ПОВТОРИТЬ ВВОД ПОЛЯ
64    ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:
65      ЗАПИСАТЬ ДАННЫЕ В файл
66      ВЫВЕСТИ: Сообщение о введенной записи
67    ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:
68      ЗАВЕРШИТЬ ЦИКЛ
69
70  ЗАКРЫТЬ файл
71  ОЧИСТИТЬ ПОТОК ВВОДА
72  ВЕРНУТЬ
73
74 функция write_license_data():
75  ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла =
file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2)
76  ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла
77
78  ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:
79    ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
80  ВЕРНУТЬ
```

```
81  ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"  
82  
83  ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:  
84      ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:  
85          ВЫВЕСТИ: Сообщение об ошибке в введенном поле  
86          ПОВТОРИТЬ ВВОД ПОЛЯ  
87      ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:  
88          ЗАПИСАТЬ ДАННЫЕ В файл  
89          ВЫВЕСТИ: Сообщение о введенной записи  
90      ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:  
91          ЗАВЕРШИТЬ ЦИКЛ  
92  
93  ЗАКРЫТЬ файл  
94  ОЧИСТИТЬ ПОТОК ВВОДА  
95  ВЕРНУТЬ
```

3

Сведения о программной реализации

3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования

Языки программирования: C++17 (GNU C Compiler 15.2.1)

Утилита сборки программы: GNU Make 4.4.1

Системы программирования: Visual Studio Code 1.106.0

3.2 Описание входных и выходных данных

Входные данные:

Пользователь выбирает опции функционирования программы и записывает данные в выбранный файл посредством ввода текста. Ввод представляет собой номер выбранной опции – число, имя файла – текст, с которым будет связано дальнейшее выполнение программы, а также последовательный ввод полей для записи в файл – текст

Структура входных данных:

данные вводятся по одному на строку, таким образом пользователь может ввести только одну опцию за раз, а заполнение полей записей файла происходит по одному на строку (одно поле – одна строка)

Выходные данные и их структура:

Пользователь имеет два вида выходных данных:

1. Консольные сообщения: представляют собой текст, необходимый пользователю для пользования программой. Сообщения включают в себя: назначение программы, опции пользования программой, информацию об успешном/неуспешном открытии файла, предупреждения, если пользователь неправильно ввел поле записи, а также сообщение о завершении программы.
2. Файлы: представляют из себя текстовые файлы, содержащие записи, введенные пользователем в порядке записи по одному на строку, имя файла либо вводится пользователем, либо присваивается стандартное программой: `car_data.txt` или `license_data.txt`, в зависимости от назначения программы.

Файл автомобильных сведений:

состоит из записей, каждая из которых содержит три поля – марка, модель и номерной знак.

Файл регистрационных сведений:

состоит из записей, каждая из которых содержит четыре поля – номерной знак, фамилия, адрес владельца и год выпуска.

3.3 Краткое описание спроектированных программных единиц

файл main.cpp:

Функция	Назначение	Аргументы	Возвращает
main	Точка входа программы	Отсутствуют	Код завершения программы. Тип: int

файл file_handler.hpp:

Подключает следующие библиотеки:

Библиотека	Назначение
cstdlib	стандартная библиотека C
cstdio	библиотека стандартного ввода/вывода C
cstring	библиотека для подключения функции memset
string	библиотека для работы с «умными» строками
regex	библиотека для работы с регулярными выражениями
filesystem	библиотека для работы с файловой системой, нужна для проверки существования файла

Содержит следующие структуры:

Структура	Назначение	Поля
CarData	Структура для хранения автомобильных сведений	license – поле хранит информацию про номерной знак автомобиля, brand – поле хранит информацию про бренд автомобиля, model – поле хранит информацию про модель автомобиля
LicenseData	Структура для хранения регистрационных сведений	license – поле хранит информацию про номерной знак автомобиля, surname – поле хранит информацию про фамилию владельца автомобиля, address – поле хранит информацию про адрес владельца автомобиля, release_year – поле хранит год выпуска автомобиля

Также содержит все прототипы функций: (подробнее о функциях ниже)

Прототип функции
<code>bool validate_filename(std::string filename)</code>
<code>bool validate_license(std::string license)</code>
<code>bool validate_brand(std::string brand)</code>
<code>bool validate_model(std::string model)</code>
<code>bool validate_surname(std::string surname)</code>
<code>bool validate_address(std::string address)</code>
<code>bool validate_release_year(std::string release_year)</code>
<code>std::pair<std::string, std::string> file_info(std::string default_filename)</code>
<code>void write_car_data()</code>
<code>void write_license_data()</code>

файл `file_handler.cpp`:

Подключает содержимое `file_handler.hpp` и содержит реализации функций

Под правильностью введенных полей, имен файлов, понимается:

1. в случае полей: это отсутствие *неподходящих символов* и *правильный* порядок элементов.
2. в случае имени файла: это *подходящее* имя файла, которое операционная система способна создать.

Функция	Назначение	Аргументы	Возвращает
<code>validate_filename</code>	Валидация имени файла	filename – имя файла. Тип: <code>std::string</code>	true если имя верно, false если неверно. Тип: <code>bool</code>
<code>validate_license</code>	Валидация поля номерной знак	license – содержимое поля номерной знак. Тип: <code>std::string</code>	true если номерной знак введен верно, иначе false. Тип: <code>bool</code>
<code>validate_brand</code>	Валидация поля бренд	brand – содержимое поля бренд. Тип: <code>std::string</code>	true если бренд введен верно, иначе false. Тип: <code>bool</code>
<code>validate_model</code>	Валидация поля модель	model – содержимое поля модель. Тип: <code>std::string</code>	true если модель введена верно, иначе false. Тип: <code>bool</code>
<code>validate_surname</code>	Валидация поля фамилия	surname – содержимое поля фамилия. Тип: <code>std::string</code>	true, если фамилия введена верно, иначе false. Тип: <code>bool</code>
<code>validate_address</code>	Валидация поля адрес	address – содержимое поля адрес. Тип: <code>std::string</code>	true, если адрес введен верно, иначе false. Тип: <code>bool</code>

Функция	Назначение	Аргументы	Возвращает
validate_release_year	Валидация поля год выпуска	release_year – содержимое поля год выпуска. Тип: std::string	true если год выпуска введен верно и подходит по временному интервалу, иначе false. Тип: bool
file_info	Спрашивает у пользователя имя файла и режим работы с ним, для последующей работы с файлом	default_file_name – стандартное имя файла, в зависимости от выбранной опции, принимает соответствующее стандартное имя файла. Тип: std::string	пару значений, первое – это имя файла, второе – это режим открытия файла. Тип: std::pair<std::string, std::string>
write_car_data	Запись введенных пользователем сведений в файл автомобильных сведений, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void
write_license_data	Запись введенных пользователем сведений в файл регистрационных сведений, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void

3.4 Список диагностических сообщений с указанием возможных причин их появления

Сообщение	Возможная причина
Введен %с программа продолжает работу\n	Введенный символ не является N,n,Q,q или «enter»(«\n»).
Введен %с. Можно ввести только опцию 1 или опцию 2. Попробуйте еще раз.«	Введенный символ не является 1 или 2.
Введена пустая строка	Пользователь ввел enter(«\n») или что-то с нулевой длиной в поле год выпуска.
Строка не полностью состоит из цифр«	Поле год, введенное пользователем, содержит что-то кроме цифр.
Год вне диапазона(%d-%d)«	Поле год, введенное пользователем, выходит за описанные временные рамки.
Введенное имя файла некорректно. Файл должен иметь вид file.txt, где file состоит из латинских символов, дефиса(-) или нижнего подчеркивания(_), а после следует его расширение .txt«	Пользователь ввел неподходящее имя файла.
Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?	Пользователь ввел имя файла, который уже существует.
Не удалось открыть файл %s\n	Файл может не открываться, если неправильно введено имя файла, или с ним уже происходит работа.
Поле введено некорректно, попробуйте еще раз\n	Поле не соответствует заданному примеру/форме или заданному множеству элементов.

4

Инструкция использования программы

Инструкция полностью соответствует указаниям на экране.

Но все же рассмотрим как пользоваться программой:

Для запуска программы пользователь должен запустить файл: ./file_h

Затем пользователь увидит следующее:

Назначение программы: Создание файлов на основе сведений, вводимых пользователем.

Что требуется записать?

- 1) Файл автомобильных сведений
 - 2) Файл регистрационных сведений
- q) Для завершения программы нажмите q.

Теперь у пользователя есть выбор, как взаимодействовать с программой.

Рассмотрим основные варианты:

Допустим пользователь ввел 1. Теперь он увидит следующее:

Введите название файла(enter=car_data.txt):

Программа ожидает от пользователя название файла, в который будут записаны сведения. Если пользователь введет enter, то он будет работать с файлом car_data.txt. Если пользователь введет любое другое имя файла, которое удовлетворяет требованиям, например file1.txt, он увидит следующие сообщения:

Файл file1.txt успешно открыт для записи.

Начинаем заполнять автомобильные сведения.

Для завершения ввода нажмите 'Q' или 'q'.

Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Но если файл file1.txt уже существует будет выведено следующее сообщение:

Файл file1.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

В таком случае программа в зависимость от ввода пользователя будет либо дописывать сведения в файл, либо заполнит его с нуля.

Теперь программа ожидает от пользователя заполнение поля бренд.

Если пользователь заполнит его без ошибок, то он перейдет к записи следующего поля. Если же пользователь совершит ошибку при вводе, то он увидит следующее сообщение:

Поле введено некорректно, попробуйте еще раз.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Это сообщение будет повторяться до тех пор, пока пользователь не введет поле правильно, или не решит завершить ввод, нажав q или Q. Как только пользователь правильно введет поле, он перейдет к заполнению следующего и так далее, пока не завершит ввод сведений, нажатием клавиши q или Q.

Стоит отметить, что данные записываются в файл при условии, что все каждое из полей было введено верно и при этом, все поля были заполнены.

Как только пользователь заполнит первую запись он увидит сообщение:

Запись 1 завершена:
бренд: Toyota
модель: Supra
номер: M976MM777RUS
Продолжаем ввод сведений.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Если пользователь введет q или Q он увидит следующее сообщение:

Желаете продолжить запись? (enter=Да/N=нет)

Теперь программа ожидает от пользователя выбор либо продолжить запись, завершить работу с программой. Если пользователь введет enter, он снова увидит сообщение о выборе опций взаимодействия с программой.

А если пользователь решит завершить работу с программой он увидит данное сообщение:

Нажмите любую клавишу для завершения программы...

Затем после нажатия любой клавиши, будет выведено следующее сообщение:

Программа завершена.

5

Листинг программной разработки

файл main.cpp:

```
1 #include "file_handler.hpp"
2
3
4 // точка входа программы
5 int main() {
6     // устанавливаем локаль UTF-8 с поддержкой кириллицы
7     SetConsoleCP(CP_UTF8);
8     SetConsoleOutputCP(CP_UTF8);
9
10    // Выводим назначение программы
11    printf("Назначение программы: Создание файлов на основе сведений, вводимых
пользователем\n");
12
13    // Начинаем взаимодействие с пользователем.
14    printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл
регистрационных сведений\n");
15    printf("q) Для завершения программы нажмите q.\n");
16
17    // Подключаем массив для буфера ввода
18    extern int BUFFER[256];
19
20    // Считываем и обрабатываем пользовательский ввод
21    while ((BUFFER[0] = getchar()) != EOF) {
22        fseek(stdin, 0, SEEK_END);
23        if (BUFFER[0] == '1' || BUFFER[0] == '2') {
24            system("cls");
25            if (BUFFER[0] == '1') write_car_data();
26            else
27                if (BUFFER[0] == '2') write_license_data();
28        } else if (BUFFER[0] == 'q' || BUFFER[0] == 'Q') {
29            break;
30        }
31        else printf("Данной опции не существует.\n");
32
33        printf("Желаете продолжить запись? (enter=Да/N=Нет)");
34        BUFFER[0] = getchar();
35        if (BUFFER[0] == 'N' || BUFFER[0] == 'n') {
36            break;
37        } else
```

```

38     if (BUFFER[0] == '\n') {
39         printf("Введен enter, программа продолжает работу\n");
40     } else printf("Введен %c, программа продолжает работу\n", BUFFER[0]);
41     system("cls");
42
43     printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл
регистрационных сведений\n");
44     printf("q) Для завершения программы нажмите q.\n");
45 }
46
47 // Сообщаем пользователю о завершении программы и
48 // ожидаем нажатие любой клавиши
49 printf("Нажмите любую клавишу для завершения программ.");
50 fseek(stdin, 0, SEEK_END);
51 scanf("%c", &BUFFER[0]);
52 printf("Программа завершена.\n");
53 // Возвращаем ОС код успешного выполнения программы.
54 return 0;
55 }

```

файл file_handler.hpp:

```

1 #ifndef FILE_HANDLER_HPP
2 #define FILE_HANDLER_HPP
3
4 #include <cstring> // библиотека для подключения memset
5 #include <cstdio> // библиотека стандартного C ввода/вывода
6 #include <cstdlib> // стандартная библиотека C
7 #include <filesystem> // библиотека для работы с файловой системой (требуется C++17)
8 #include <regex> // библиотека для работы с регулярными выражениями
9 #include <string> // библиотека для работы с "умными" строками
10 #include <windows.h> // для изменения локали на машинах windows
11
12 // Структура для временного хранения автомобильных сведений
13 struct CarData {
14     std::string license;
15     std::string brand;
16     std::string model;
17 };
18
19 // Структура для временного хранения регистрационных сведений
20 struct LicenseData {
21     std::string license;
22     std::string surname;
23     std::string address;
24     std::string release_year;
25 };
26
27 // Ниже указаны прототипы функций,
28 // подробнее о них в файле реализаций file_handler.cpp
29
30 // прототип функции очистки ввода(костыль)
31 void clear_stdin();
32
33 // прототипы функций валидации
34 bool validate_filename(std::string filename);
35 bool validate_license(std::string license);
36 bool validate_brand(std::string brand);
37 bool validate_model(std::string model);
38 bool validate_surname(std::string surname);
39 bool validate_address(std::string address);
40 bool validate_release_year(std::string release_year);

```

```

41
42 // прототипы функций, связанных с работой с файлами
43 std::pair<std::string, std::string> file_info(std::string default_filename);
44 void write_car_data();
45 void write_license_data();
46
47 #endif

```

файл file_handler.cpp

```

1 #include "file_handler.hpp"
2
3
4 // буффер ввода
5 char BUFFER[256];
6
7 // Инициализируем основные стандартные параметры
8 std::string DEFAULT_FILENAME_1 = "car_data.txt";
9 std::string DEFAULT_FILENAME_2 = "license_data.txt";
10 int LOWER_BOUND_YEAR = 1960;
11
12 // Стоит отметить, что почти все функции валидации используют
13 // регулярные выражения, сделано это для упрощения проверок
14
15 // функция валидации имени файла
16 bool validate_filename(std::string filename) {
17     std::regex regex_filename{"[a-zA-Z0-9_-]+\\.txt"};
18     return std::regex_match(filename, regex_filename);
19 }
20
21 // функция валидации поля "номерной знак"
22 bool validate_license(std::string license) {
23     // M976MM777RUS
24     std::regex pattern{"[АВЕКМНОРСТУХ][0-9]{3,3}[АВЕКМНОРСТУХ]{2,2}[0-9]{2,3}RUS"};
25     return regex_match(license, pattern);
26 }
27
28 // функция валидации поля "бренд"
29 bool validate_brand(std::string brand) {
30     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЬьЪьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.! ]+"};
31     return regex_match(brand, pattern);
32 }
33
34 // функция валидации поля "модель"
35 bool validate_model(std::string model) {
36     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЬьЪьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.!() ]+"};
37     return regex_match(model, pattern);
38 }
39
40 // функция валидации поля "фамилия владельца"
41 bool validate_surname(std::string surname) {
42     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЬьЪьЭэЮюЁёЫыА-Яа-яА-Za-z]+"};
43     return regex_match(surname, pattern);
44 }
45
46 // функция валидации поля "адрес владельца"
47 bool validate_address(std::string address) {
48     std::regex address_pattern{"[ФфРрТтУуХхЦцЧчШшЩщЬьЪьЭэЮюЁёЫыА-Яа-яА-Za-z0-9!_-. ]+"};
49     return regex_match(address, address_pattern);
50 }
51
52 // функция валидации поля "год выпуска"
53 // Здесь проверка не содержит регулярных выражений

```



```

54 // Проверка заключается в попытке перевести пользовательский ввод в число,
55 // а затем в проверке подходит ли число временному диапазону
56 bool validate_release_year(std::string release_year) {
57     bool result=true;
58     if (release_year.size()==0) {
59         printf("Введена пустая строка.\n");
60         return false;
61     }
62     int year=0;
63     int ten = 1;
64     for (int i=release_year.size()-1; i>=0; --i) {
65         if (!isdigit(release_year[i])) {
66             printf("Строка не полностью состоит из цифр.\n");
67             return false;
68         }
69         year += (release_year[i]-'0')*ten;
70         ten *= 10;
71     }
72     std::time_t t = std::time(nullptr);
73     std::tm *const pTInfo = std::localtime(&t);
74     int current_year = 1900 + pTInfo->tm_year;
75     if (!(year >= LOWER_BOUND_YEAR && year <= current_year) || year <= 0) {
76         printf("Год вне диапазона(%d-%d)\n", LOWER_BOUND_YEAR, current_year);
77         result = false;
78     }
79     return result;
80 }
81
82 // функция для определения имени и режима открытия файла
83 // Она спрашивает у пользователя имя файла и, если
84 // ОС позволяет использовать данное имя файла, то
85 // файл будет с заданным пользователем именем файла,
86 // иначе надо будет вводить имя файла, пока оно не удовлетворит критериям ОС.
87 std::pair<std::string, std::string> file_info(std::string default_file_name) {
88     std::pair<std::string, std::string> file_info;
89     std::string file_name;
90     printf("Введите название файла(enter=%s): ", default_file_name.c_str());
91     while (scanf("%255[^\n]s", BUFFER)) {
92         fseek(stdin, 0, SEEK_END);
93         file_name = BUFFER;
94         memset(BUFFER, 0xFF, 256);
95         BUFFER[0]='\0';
96         if (validate_filename(file_name)) break;
97         else {
98             printf("Введенное имя файла некорректно. Файл должен иметь вид file.txt,
99 \n");
100             printf("где file состоит из латинских символов, дефиса(-) или нижнего
101 подчеркивания(_),\n");
102             printf("а после следует его расширение .txt\n");
103             printf("Введите название файла(enter=%s): ", default_file_name.c_str());
104         }
105     }
106     if (file_name.size()==0 || BUFFER[0]=='\0') file_name = default_file_name;
107     file_info.first = file_name;
108     // Далее идет проверка существует ли данный файл, если да, то необходимо спросить
109     // у пользователя,
110     // как именно предстоит работать с ним.
111     if (std::filesystem::exists(file_name)) {
112         printf("Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с
113 начала(enter=Да/N=Нет)?", file_name.c_str());
114         fseek(stdin, 0, SEEK_END);
115         while ((BUFFER[0] = getchar()) != EOF) {

```

```

112         if (BUFFER[0] == '\n') {
113             file_info.second = "a";
114             break;
115         } else
116         if (BUFFER[0] == 'n' || BUFFER[0] == 'N') {
117             file_info.second = "w";
118             break;
119         } else printf("Введено '%c' можно ввести только enter или N\n", BUFFER[0]);
120     }
121 } else file_info.second = "w";
122 fseek(stdin, 0, SEEK_END);
123 memset(BUFFER, 0xFF, 256);
124 BUFFER[0]='\0';
125 return file_info;
126 }
127
128 // функция записи автомобильных сведений в файл
129 void write_car_data() {
130     std::pair<std::string, std::string> file_information =
131     file_info(DEFAULT_FILENAME_1);
132     std::string file_name = file_information.first;
133     FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());
134
135     // проверка на открытие файла
136     if (!file) {
137         printf("Не удалось открыть файл %s\n", file_name.c_str());
138         return;
139     }
140     printf("Файл %s успешно открыт для записи\n", file_name.c_str());
141
142     int field_index=0;
143     CarData temp = {};
144     printf("Начинаем заполнять автомобильные сведения.\nДля завершения ввода нажмите
145     'Q' либо 'q'.\n");
146     std::string field_names[3] = {"бренд (символы: A-ЯЁа-яёA-Za-z0-9-_.! )", "модель
147     (символы: A-ЯЁа-яёA-Za-z0-9-_.!() )", "номерной знак (пример: M976MM777RUS)"};
148     printf("Введите поле %s:\n", field_names[field_index].c_str());
149     bool filled = false;
150     int note_number = 0;
151
152     // Ниже идет последовательная запись полей и соответствующих им записей в файл
153     // Если поле не проходит валидацию, то оно не будет занесено в файл,
154     // пока не пройдет валидацию.
155     while (scanf("%255[^\n]s", BUFFER) != EOF) {
156         if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
157             printf("Введен %s. Ввод сведений закончен\n", BUFFER);
158             fseek(stdin, 0, SEEK_END);
159             memset(BUFFER, 0xFF, 256);
160             BUFFER[0]='\0';
161             break;
162         }
163         fseek(stdin, 0, SEEK_END);
164         switch (field_index) {
165             case 0: {
166                 temp.brand.assign(BUFFER);
167                 memset(BUFFER, 0xFF, 256);
168                 BUFFER[0]='\0';
169                 if (validate_brand(temp.brand))
170                     field_index=(field_index+1)%3;
171                 else printf("Поле введено некорректно, попробуйте еще раз\n");
172                 break;
173             }
174         }
175     }

```

```

171         case 1: {
172             temp.model.assign(BUFFER);
173             memset(BUFFER, 0xFF, 256);
174             BUFFER[0]='\0';
175             if (validate_model(temp.model))
176                 field_index=(field_index+1)%3;
177             else printf("Поле введено некорректно, попробуйте еще раз\n");
178             break;
179         }
180         case 2: {
181             temp.license.assign(BUFFER);
182             memset(BUFFER, 0xFF, 256);
183             BUFFER[0]='\0';
184             if (validate_license(temp.license)) {
185                 field_index = (field_index+1)%3;
186                 filled = true;
187             } else {
188                 printf("Поле введено некорректно, попробуйте еще раз\n");
189                 printf("Первый элемент должен быть 1 из [АВЕКМНОРСТУХ],\n");
190                 printf("Следующие 3 элемента это цифры: [0-9]\n");
191                 printf("Следующие 2 элемента должны быть из [АВЕКМНОРСТУХ]\n");
192                 printf("Следующие 2-3 элемента это цифры: [0-9]\n");
193                 printf("После всего этого следует строчка: RUS\n");
194                 printf("Номерной знак не должен содержать никаких разделителей\n");
195             }
196             break;
197         }
198     }
199     if (filled) {
200         fprintf(file, "%s %s %s\n", temp.brand.c_str(), temp.model.c_str(),
temp.license.c_str());
201         printf(
202             "Запись #%d завершена:\nбренд: %s\nмодель: %s\nномерной знак: %s\n",
203             ++note_number,
204             temp.brand.c_str(),
205             temp.model.c_str(),
206             temp.license.c_str()
207         );
208         printf("Продолжаем запись сведений\n");
209         temp = {};
210         filled = false;
211         memset(BUFFER, 0xFF, 256);
212         BUFFER[0]='\0';
213     }
214     printf("Введите поле %s:\n", field_names[field_index].c_str());
215 }
216 fclose(file);
217 // Здесь происходит очистка потока ввода от EOF
218 clearerr(stdin);
219 return;
220 }
221
222 // функция записи регистрационных сведений в файл
223 // она полностью аналогична функции write_car_data(),
224 // лишь за тем исключением что записывает другие поля
225 void write_license_data() {
226     std::pair<std::string, std::string> file_information =
file_info(DEFAULT_FILENAME_2);
227     std::string file_name = file_information.first;
228     FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());
229
230     if (!file) {
231         printf("Не удалось открыть файл %s\n", file_name.c_str());

```

```

232     return;
233 }
234 printf("Файл %s успешно открыт для записи\n", file_name.c_str());
235
236 int field_index=0;
237 LicenseData temp = {};
238
239 printf("Начинаем заполнять регистрационные сведения.\nДля завершения ввода нажмите
'Q' либо 'q'.\n");
240 std::string field_names[4] = {"номерной знак (пример: M976MM777RUS)", "фамилия
владельца (символы: А-ЯЁа-яёА-Za-z)", "адрес владельца (символы: А-Za-zА-ЯЁа-яё0-9!
_-. )", "год выпуска (символы: 0-9)"};
241 printf("Введите поле %s:\n", field_names[field_index].c_str());
242 bool filled = false;
243 int note_number = 0;
244 while (scanf("%255[^\n]s", BUFFER) != EOF) {
245     if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
246         printf("Введен %s. Ввод сведений закончен\n", BUFFER);
247         fseek(stdin, 0, SEEK_END);
248         break;
249     }
250     fseek(stdin, 0, SEEK_END);
251     switch (field_index) {
252         case 0: {
253             temp.license.assign(BUFFER);
254             memset(BUFFER, 0xFF, 256);
255             if (validate_license(temp.license)) {
256                 field_index = (field_index+1)%4;
257             } else {
258                 printf("Поле введено некорректно, попробуйте еще раз\n");
259                 printf("Первый элемент должен быть 1 из [АВЕКМНОРСТУХ],\n");
260                 printf("Следующие 3 элемента это цифры: [0-9]\n");
261                 printf("Следующие 2 элемента должны быть из [АВЕКМНОРСТУХ]\n");
262                 printf("Следующие 2-3 элемента это цифры: [0-9]\n");
263                 printf("После всего этого следует строка: RUS\n");
264                 printf("Номерной знак не должен содержать никаких разделителей\n");
265             }
266             break;
267         }
268         case 1: {
269             temp.surname.assign(BUFFER);
270             memset(BUFFER, 0xFF, 256);
271             BUFFER[0]='\0';
272             if (validate_surname(temp.surname)) {
273                 field_index = (field_index+1)%4;
274             } else printf("Поле введено некорректно, попробуйте еще раз\n");
275             break;
276         }
277         case 2: {
278             temp.address.assign(BUFFER);
279             memset(BUFFER, 0xFF, 256);
280             BUFFER[0]='\0';
281             if (validate_address(temp.address)) {
282                 field_index = (field_index+1)%4;
283             } else printf("Поле введено некорректно, попробуйте еще раз\n");
284             break;
285         }
286         case 3: {
287             temp.release_year.assign(BUFFER);
288             memset(BUFFER, 0xFF, 256);
289             BUFFER[0]='\0';
290             if (validate_release_year(temp.release_year)) {

```

```
291         field_index = (field_index+1)%4;
292         filled = true;
293     } else printf("Поле введено некорректно, попробуйте еще раз\n");
294     break;
295 }
296
297 }
298 if (filled) {
299     fprintf(file, "%s %s %s %s\n", temp.license.c_str(), temp.surname.c_str(),
temp.address.c_str(), temp.release_year.c_str());
300     printf(
301         "Запись #%d завершена:\nномерной знак: %s\nфамилия владельца:
%s\nадрес: %s\nгод выпуска: %s\n",
302         ++note_number,
303         temp.license.c_str(),
304         temp.surname.c_str(),
305         temp.address.c_str(),
306         temp.release_year.c_str()
307     );
308
309     printf("Продолжаем запись сведений\n");
310     temp = {};
311     filled = false;
312 }
313 printf("Введите поле %s:\n", field_names[field_index].c_str());
314 }
315 fclose(file);
316 // очищаем поток от EOF
317 clearerr(stdin);
318 memset(BUFFER, 0xFF, 256);
319 BUFFER[0]='\0';
320 return;
321 }
```

6

Результаты тестирования

Для демонстрации работы программы заполним файл car_data.txt и выведем его содержимое на экран:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
1
Введите название файла(enter=car_data.txt):
Файл car_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл car_data.txt успешно открыт для записи
Начинаем заполнять автомобильные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ):
Dodge
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ):
Charger 1969
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Запись #1 завершена:
бренд: Dodge
модель: Charger 1969
номерной знак: M977MM777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ):
Toyota
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ):
Supra
Введите поле номерной знак (пример: M976MM777RUS):
M976MM766RUS
Запись #2 завершена:
бренд: Toyota
модель: Supra
номерной знак: M976MM766RUS
```

Рис. 1. Пример заполнения первых 2-х записей.

Введем еще одну запись и завершим ввод сведений в файл.

```
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
Мерседес
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ) :
Модель S
Введите поле номерной знак (пример: М976ММ777RUS):
М976ММ777RUS
Запись #3 завершена:
бренд: Мерседес
модель: Модель S
номерной знак: М976ММ777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
q
Введен q. Ввод сведений закончен
```

Рис. 2. Пример заполнения еще одной записи.

Опять видим основные опции программы. Завершим работу с программой.

```
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
q
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 3. Снова основное меню.

Выведем на экран содержимое файла car_data.txt.

```
→ cat car_data.txt
Dodge Charger 1969 М977ММ777RUS
Toyota Supra М976ММ766RUS
Мерседес Модель S М976ММ777RUS
```

Рис. 4. Содержимое car_data.txt.

Можно заметить, что информация записалась в файл без каких либо проблем.

Теперь запишем информацию в файл license_data.txt:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt): license_data.txt
Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Введите поле фамилия владельца (символы: A-ЯЁа-яёA-Za-z):
Pushkin
Введите поле адрес владельца (символы: A-Za-zA-ЯЁа-яё0-9!_-. ):
Ул. Новые Черемушки
Введите поле год выпуска (символы: 0-9):
2008
Запись #1 завершена:
номерной знак: M977MM777RUS
фамилия владельца: Pushkin
адрес: Ул. Новые Черемушки
год выпуска: 2008
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 5. Пример заполнения license_data.txt.

Теперь выведем содержимое файла на экран.

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
```

Рис. 6. Содержимое license_data.txt.

Теперь допишем информацию в license_data.txt.

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt):
Файл license_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M976CC777RUS
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Za-z):
&
Поле введено некорректно, попробуйте еще раз
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Za-z):
Nikitin
Введите поле адрес владельца (символы: А-Za-zА-ЯЁа-яё0-9!_-. ):
Улица Профсоюзная д.26
Введите поле год выпуска (символы: 0-9):
1977
Запись #1 завершена:
номерной знак: M976CC777RUS
фамилия владельца: Nikitin
адрес: Улица Профсоюзная д.26
год выпуска: 1977
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 7. Пример дозаписи сведений в license_data.txt.

Стоит отметить, что при заполнении поля фамилия была совершена ошибка. Программа заметила, что поле введено некорректно, и потребовала перезаписать поле.

Теперь посмотрим содержимое license_data.txt

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
M976CC777RUS Nikitin Улица Профсоюзная д.26 1977
```

Рис. 8. Содержимое license_data.txt после дозаписи.

В результате тестирования можно отметить, что программа функционирует корректно.